

Tick & Taka — Product & Technical Spec

Oct 4, 2026 · @Bipul Hf

Positioning

Tick & Taka is a private, single-user mobile app that runs my days and my money from one screen. "Tick" is time, "Taka" is money; the name is a working title.

The problem. Time apps and money apps live apart, so neither knows what the other sees. Logging in both feels like a chore, so both get abandoned within weeks.

The promise. Open the app and see today: what to do next, and how much I can safely spend. Log anything in under five seconds.

Who it is for. One person juggling several workstreams at once: two remote jobs, teaching, research and home. There are no other users, so there is no sharing, onboarding or team features.

Platform: Android only. The app is built, tested and installed as an APK for Android alone, with no iOS version. That frees it to use Android-only features without workarounds: reading SMS, home-screen widgets, long-press app shortcuts ("Add expense", "Start focus"), and action buttons on notifications.

Guiding principles

- **Capture first, organise later.** Any task, expense or time entry takes one tap plus typing. Sorting happens in the evening review, not at capture.
- **Today over everything.** The home screen answers two questions: "what now?" and "can I afford it?"
- **Reward, never guilt.** Progress gets a small celebration. A missed day gets a shrug, never a red warning.
- **Honest numbers.** Account balances and tracked hours always reconcile. No rounded or estimated totals presented as facts.
- **Every feature earns its place.** A feature I don't touch for a month gets hidden or removed.
- **My data, my server.** Everything lives on my own VPS and exports to a single file anytime.

What it is not: a bank-sync aggregator, a team tool, or a full calendar replacement.

Benchmark: what the leading apps get right

Twelve widely used apps each do one thing better than anyone else; Tick & Taka copies that one thing from each and skips the rest. Reviewers keep repeating the same lesson: the best budget app is the one you keep using, and feature-packed apps like TickTick bury their best ideas under options most people never touch.

App	Best known for	What we copy	Source
YNAB	Zero-based budgeting: every unit of income gets a job before it's spent; a loan payoff simulator	Payday plan, debt payoff forecast	NerdWallet
Monarch Money	Flex budgeting in three buckets (fixed, non-monthly recurring, flexible); weekly spending recaps	Three-bucket budgets, weekly recap	NerdWallet
PocketGuard	"Left to spend" after bills and goals; Pace alerts when spending runs ahead of the month	Pace alert on top of safe-to-spend	NerdWallet
Copilot Money	AI categorisation that learns your patterns; spots forgotten subscriptions; budget rollover	Learn-from-corrections categories, subscription spotter	NerdWallet
Money Manager (Realbyte)	20M+ downloads; calendar of daily spending, built-in calculator, home-screen widget, automatic double-entry	Money calendar, calculator keypad, widget	App Store
Wallet by BudgetBakers	Planned payments; shopping lists with item prices and reminders	Priced shopping list	BudgetBakers, Finder

App	Best known for	What we copy	Source
Money Lover	Events: one budget for a trip or celebration, grouping spending from every wallet	Events	CompareHero
Todoist	Fastest natural-language entry, including recurrences like "every other Tuesday"; Karma daily goals, days off and vacation mode	Recurrence in plain words, daily goal, vacation mode	Durple , Todoist help
TickTick	Tasks, calendar, Pomodoro and habits in one app; Eisenhower matrix; focus statistics	Eisenhower view, focus statistics	TidBITS , Toodledo
Things 3	A strict Today with a This Evening section; Someday; Logbook; "when I'll do it" kept separate from the deadline	Evening, Someday, Logbook, do-date vs. deadline	Cultured Code
Structured	One timeline of start times and durations that shows whether the planned day actually fits; inbox; energy levels	"Does my day fit?" bar, energy tag	Cool Curation
Forest	A tree grows while you stay focused and dies if you leave the app	A softened version: Tiki's plant droops instead of dying	ClickUp

Research done on 4 October 2026 from the pages linked above.

Design language

The app should feel like a cheerful pocket companion: soft shapes, warm colours, bouncy feedback, and zero guilt. Playfulness is there to make logging feel rewarding, so it must never slow a task down.

Mascot: Tiki. A round coin with a little clock face. Tiki reacts to the day: proud when the top three tasks are done, relaxed when spending is under budget, sleepy after 11 pm. When I overspend, Tiki looks calm and offers a tip, never sad or angry. Start with 6–8 SVG expressions swapped with Reanimated; move to Rive animations only if it feels flat.

Colour palette

Token	Light	Dark	Used for
Mango (primary)	#FFB547	#FFC266	Main buttons, quick-add, highlights
Sky (time)	#5B8CFF	#7AA2FF	Tasks, focus timer, calendar
Mint (money in)	#2EC4A0	#4FD8B5	Income, under budget, goals
Coral (money out)	#FF7A6B	#FF9385	Expenses, over budget (soft, not alarm red)
Grape (habits)	#A57BFF	#B996FF	Habits, streaks, routines
Background	#FFF8EE	#16151C	Screen background
Card	#FFFFFF	#22202B	Cards and sheets
Text	#23202B	#F4F1EA	Body text

Time features lean blue, money features lean green and coral, so I always know which half of the app I'm in at a glance.

Typography. Nunito for all UI text: rounded, friendly and very legible. Every amount and timer uses tabular digits (`fontVariant: ['tabular-nums']`) so numbers don't jiggle while they animate. Add Hind Siliguri later if I want Bangla labels. Amounts always show the ₳ sign.

Motion and feel

- Springs, not linear easing, for every transition (Reanimated).
- Ticking a task: the checkbox pops, a light haptic fires, a spark flies to the score.
- Big wins only get confetti: all top three done, a savings goal reached, a 30-day streak. Rarity keeps it special.

- Balances and totals count up to their new value instead of jumping.
- Respect the phone's "reduce motion" setting.

Light gamification. Sparks are points earned by finishing tasks, completing focus sessions and logging expenses on the day they happen. Sparks raise a level that unlocks cosmetic rewards only: themes, Tiki outfits, app icons. Habit and logging streaks get 2 freeze days per month, so one bad day never breaks a long run. No leaderboards, no penalties.

Interaction rules

- Primary actions sit in the bottom half of the screen, inside thumb reach.
- Tap targets are at least 48 dp.
- Swipe right to complete or check off; swipe left to snooze or edit.
- Creating and editing happen in bottom sheets, never on a new full screen.
- Deleting shows an Undo snackbar instead of an "Are you sure?" dialog.
- Every empty state shows Tiki plus exactly one suggested action.
- Copy is short and friendly: "Safe to spend today: ₮640", not "Remaining daily discretionary budget".
- Dark mode follows the system setting.

App structure

Five slots in the bottom bar, with quick-add in the centre so capture is always one thumb-tap away.

Tab	What lives there	The question it answers
Today	Agenda, top three, safe-to-spend, habits, running timer	What now, and can I afford it?
Plan	Task inbox, projects, week planner, calendar view	What's coming and what's waiting?
+ (quick-add)	One sheet that logs a task, expense, income or time entry	—
Money	Accounts, transactions, budgets, bills, goals, debts	Where is my money and where is it going?

Tab	What lives there	The question it answers
Review	Insights, weekly review, AI coach, settings	How am I doing, and what should change?

Today screen, top to bottom

- 1. Greeting, date and Tiki's mood for the day.
- 2. Safe-to-spend card: how much I can spend today without breaking this month's budgets, plus what I've spent so far today.
- 3. Top three: the three tasks I picked for today, with big checkboxes.
- 4. Timeline: time blocks, tasks with a time, and bills due today on one vertical line, topped by a "does my day fit?" bar.
- 5. Habits row: round chips I tap to check off.
- 6. Running timer: a sticky bar while a focus session or time entry is running.
- 7. This Evening: after-work tasks, kept apart from the working day, followed by the next three bills or reminders.

Quick-add. One text field, one Save button. The app works out what I meant, and type chips above the field let me override it.

I type	It becomes
lunch 250	Expense ₹250 in Food, from my default account
+45000 salary	Income ₹45,000
call bank tomorrow 5pm	Task due tomorrow with a 5 pm reminder
2h thesis writing	Time entry of 2 hours under Research
rickshaw 60 bkash	Expense ₹60 in Transport, from the bKash account

Simple patterns are parsed on the phone instantly, even offline. Anything ambiguous goes to the AI parser on the backend (see AI features). A preview line shows what will be saved before I tap Save.

Time features

Everything time-related hangs off **areas**, the big buckets of my life: Job 1, Job 2, Teaching, Research, Home and Personal (renameable). Tasks, time entries and money all carry an area, which is what lets the two halves of the app talk to each other.

Feature	What it does	Why it exists
Areas and projects	Areas hold projects; projects hold tasks. Each area has a colour and an emoji.	One tag that links tasks, hours and money, so I see where my life actually goes.
Inbox	Every quick-add task lands here unsorted.	Capture without deciding. Sorting waits for the evening.
Tasks	Title, notes, due date and time, priority (low, normal, high), subtasks, estimate, reminder, repeat rule.	One trusted list instead of tasks scattered across chats and notes.
Top three	Each day I pick three tasks that matter most. They sit at the top of Today.	A day I can actually finish. Three is small enough to say no to the rest.
Week planner	Drag tasks onto days and into time blocks on a timeline.	Plans that fit real hours, not wishful lists.
Focus timer	Pomodoro-style timer (default 25 min work, 5 min break) linked to a task. Keeps running in the background with a notification.	Starting is the hardest part. One tap starts work and records the time automatically.
Time tracking	Start/stop timer or manual entry, per area and project, with a billable flag.	Know exactly how many hours each job and course takes each week.
Habits	Daily, weekly or "N times a week" targets, optionally with a count (e.g. 8 glasses). Streaks with 2 freeze days a month.	Small repeated actions are what change things; streaks make them stick.
Routines	Ordered checklists such as Morning or Shutdown, with optional per-step timers.	Removes decisions from the start and end of the day.

Feature	What it does	Why it exists
Calendar view	Month and week views showing tasks, time blocks and bills together.	Spot crowded days and collisions before they happen.
Overdue rescue	One tap moves all overdue tasks to today, tomorrow or back to the inbox.	Stops overdue tasks piling up into a guilt list.

Two rules keep this half light: tasks have no more than one level of subtasks, and there are no tags beyond area and project.

Money features

The money half answers three questions every day: what do I have, where is it going, and what's safe to spend today. Default currency is BDT (₳); any account can hold another currency.

Feature	What it does	Why it exists
Accounts	Cash, bank accounts, bKash, Nagad, cards and savings, each with a currency, opening balance and icon.	Balances in the app match the real world, wallet included.
Transactions	Expense, income or transfer, with amount, account, category, area, note, date and an optional receipt photo. Transfers can carry a fee (e.g. a bKash cash-out charge).	The single source of truth for every taka in and out.
Categories	Emoji categories, at most two levels deep (Food › Eating out). Sensible defaults on first launch.	Quick to pick on a phone, detailed enough for useful reports.
Budgets	Monthly limit per category, with optional rollover of unspent money. Split into three buckets: fixed (rent, bills), non-monthly (yearly renewals, Eid shopping) and flexible (food, fun).	Spending decisions get made before the month, not after.

Feature	What it does	Why it exists
Safe to spend	A daily number on the Today screen, worked out from flexible budgets only.	Turns a monthly budget into one decision I can make at a shop counter.
Bills and subscriptions	Recurring payments with due dates and a reminder 2 days before. Tapping "Paid" creates the transaction.	No late fees and no forgotten subscriptions.
Expected income	Salaries and other income on a schedule. Marking one received logs it, converting foreign currency at the rate I enter.	Know what's coming in, and record what a foreign-currency salary really became in taka.
Savings goals	Jars with a target and optional deadline. The app suggests a monthly amount to stay on pace.	Saving becomes a visible, finishable project.
Debts and lending	Who owes me and whom I owe, with partial repayments and optional reminders.	Small personal loans are easy to forget and awkward to raise; this keeps them straight.
Balance check	Enter an account's real balance; the app logs an adjustment for the difference.	Keeps numbers honest when I miss a log, without hunting for the gap.
Reports	Spending by category, income vs. expense by month, net worth over time, spending by area.	Patterns I can act on at the monthly review.

Safe to spend today = flexible budget left this month ÷ days left in the month, including today. Fixed bills are excluded because they're already planned for.

All amounts are stored as whole numbers of the smallest unit (poisha for BDT, cents for EUR), never as decimals.

SMS auto-capture

The app reads bank and mobile-wallet SMS on my phone and turns each transaction message into a ready-made card in an **SMS review panel**. Nothing is saved until I tap Add, so a wrong parse never pollutes my numbers.

How a message becomes a transaction

1. **Permission, once.** On first use, an explainer screen asks for `READ_SMS` and `RECEIVE_SMS`.
2. **Only allowed senders.** In Settings › SMS sources I pick which sender IDs count (bKash, Nagad, my banks) and map each to an account. Every other message is ignored, as is anything containing "OTP", "code" or "PIN".
3. **Scan.** Each time the app opens, it reads allowed messages that arrived since the last scan. A background receiver also catches new SMS while the app is closed.
4. **Parse on the phone.** A template per sender pulls out amount, direction, fee, balance after, transaction ID and the other party. Templates are built from 2–3 of my own real messages per sender, pasted in during setup.
5. **AI fallback.** A message no template matches goes to `/ai/parse` with account numbers, phone numbers and names masked. The reply is a draft like any other.
6. **No duplicates.** Each message is fingerprinted by its transaction ID, or by sender + time + amount when there is no ID. A fingerprint is never offered twice.
7. **Learn from edits.** If I change a card's category, the rule is saved to `category_rules`, so the same merchant comes pre-filled next time.

How wording maps to records

The SMS says	It becomes
Money received, cash in, credited	Income to the mapped account, or a transfer if it came from another of my accounts
Payment, send money, purchase, debited	Expense from the mapped account
Cash out	Transfer from the wallet to the Cash account, with the fee logged as an expense
A fee or charge line	The transaction's <code>fee_minor</code>
Balance after the transaction	Compared with the app's balance for that account

The review panel

- A "3 new from SMS" banner appears on Today, with a matching badge on the Money tab. Tapping either opens the panel as a bottom sheet.

- Each card shows: sender logo and mapped account; the amount, large, in coral for money out or green for money in; a type chip (Expense, Income, Transfer); a category chip pre-filled from my rules; a note pre-filled with the merchant or person; and the time received.
- Three buttons per card: **Add** (primary), **Edit** (opens the quick-add sheet pre-filled) and **Ignore**. Swipe right to add, left to ignore.
- **Add all** appears when every card has a confident category, so a day's five bKash payments take one tap.
- When the SMS balance doesn't match the app's balance after adding, the card offers a one-tap balance check: "App says ₦4,120, SMS says ₦3,870 — fix it?"
- Ignored cards stay in a collapsed "Ignored" list for 30 days, so a mistaken swipe can be undone.
- A new transaction SMS triggers a notification such as "bKash · ₦500 payment — add as expense?" with **Add** and **Review** action buttons.
- The daily shutdown routine starts with any cards still waiting.

What has to change in the build

- Expo Go can't load custom native code, so the app moves to a **development build** (`expo-dev-client`) built with EAS.
- A small local Expo module, `sms-reader`, written in Kotlin, provides two things: `getMessagesSince(timestamp, senders)` reading `content://sms/inbox`, and an `onSmsReceived` event from a broadcast receiver.
- `app.json` declares the `READ_SMS` and `RECEIVE_SMS` Android permissions.
- The app stays a sideloaded APK. Google Play only allows SMS permissions for default SMS apps, so it never goes to the Play Store, which suits a single-user app anyway.

Data and API

- New table `sms_imports`: fingerprint, sender, received_at, parsed amount and direction, status (pending, added, ignored) and transaction_id.
- `POST /sms-imports` sends a batch of parsed cards and returns which fingerprints the server already knows, so reinstalling the app never re-offers old messages.
- `PATCH /sms-imports/:fingerprint` records Add or Ignore.
- **Raw SMS text never leaves the phone**, except the masked text in an AI fallback. The server stores parsed fields and the fingerprint only.

Setup, step by step

1. In `apps/mobile`, run `bunx expo install expo-dev-client`.
2. Run `bunx create-expo-module@latest --local sms-reader`.
3. When it asks for a name and package, accept the suggested values.
4. Open `app.json`.
5. Inside the `"android"` object, add `"permissions": ["READ_SMS", "RECEIVE_SMS"]`.
6. Write the Kotlin code for `getMessagesSince` and `onSmsReceived` in `modules/sms-reader/android/`.
7. Run `bunx eas-cli build -p android --profile development`.
8. Download the APK from the link the build prints.
9. Install it on the phone.
10. On the computer, run `bunx expo start --dev-client`.
11. Open the installed app (not Expo Go); it connects to the dev server.
12. Tap Allow when the app asks for SMS access.
13. If the SMS permission is greyed out, open phone Settings › Apps › Tick & Taka, tap the `:` menu, choose **Allow restricted settings**, then grant SMS again. Recent Android versions add this extra step for sideloaded apps.
14. In Settings › SMS sources, add each sender and paste 2–3 sample messages for its template.

Scan-on-open and the background receiver with notifications are built together.

Where time meets money

These features only work because both halves share one database and one set of areas. They are the reason this is one app instead of two.

Feature	What it does	Why it exists
Effective hourly rate	Per area each month: income tagged to that area ÷ hours tracked in it.	Shows which job or gig actually pays best for my time.
Cost in hours	When I log a large expense, a small line shows its price in work hours (e.g. "≈ 6.5 hours of work"). Can be turned off.	Makes a purchase feel real before it becomes a habit.

Feature	What it does	Why it exists
One timeline	Bills, paydays and debt reminders appear on the Today timeline and in the calendar beside tasks.	Money deadlines are deadlines too.
Area dashboard	For each area: hours this week, money in and out this month, open tasks.	One view of how each part of my life is going.
Goals become tasks	A savings goal with a deadline creates a monthly "move ₺X to the jar" task.	Turns a money intention into a concrete action.
Daily shutdown	A 2-minute evening routine: log missed spending, check habits, pick tomorrow's top three. Tiki says good night.	Closes the day so nothing leaks, and makes tomorrow start with a plan.
Weekly review	A 5-minute guided flow on Sunday: hours by area vs. plan, spending vs. budget, habit streaks, wins, then next week's focus.	The habit that keeps the whole system honest and useful.
Monthly review	Budgets vs. actual, net worth change, hourly rates by area, then adjust next month's budgets.	Bigger corrections happen once a month, calmly.

Borrowed features, with real use cases

Twenty-three proven features join the plan, each tied to a moment in my actual week. None adds a new tab; every one lives inside a screen that already exists.

Money

Feature	Copied from	How I'll use it
Payday plan	YNAB	My salary lands. A 2-minute sheet opens and I split it across rent, bills, family, the savings jar and flexible spending until ₺0 is left unassigned.

Feature	Copied from	How I'll use it
Three-bucket budgets	Monarch	Rent and internet are fixed . The yearly domain renewal and Eid shopping are non-monthly , so I set aside a slice every month. Food, rickshaw and fun are flexible , and only these drive safe-to-spend.
Pace alert	PocketGuard	Mid-month, Food is spending faster than the days passing. Tiki sends one calm heads-up, never a stream of warnings.
Calculator keypad	Money Manager	Splitting a dinner bill: I type 1850/3 in the amount field and it saves ₹617.
Money calendar	Money Manager	A month grid shows each day's spending. I tap a spike on the 15th and see it was a wedding gift plus a CNG ride.
Home-screen widget	Money Manager	Log "cha 20" without opening the app; the same widget shows safe-to-spend for today.
Debt payoff forecast	YNAB	On a debt, I enter a monthly amount and see the month it will be cleared.
Events	Money Lover	A "Sylhet trip" or "Eid" event gets its own budget and collects spending from cash, bKash and card. Afterwards I see the full trip cost in one place.
Priced shopping list	Wallet	Before the bazar I list items with estimated prices and see the total. Ticking them off at checkout turns the list into one expense.
Learn-from-corrections categories	Copilot	When I move "Foodpanda" from Shopping to Food once, every future Foodpanda note goes to Food.
Subscription spotter	Copilot	The monthly review flags charges that repeat at the same amount, so a forgotten subscription becomes a tracked bill or gets cancelled.

Feature	Copied from	How I'll use it
Weekly recap card	Monarch	Friday morning: this week's spend vs. last week, the top category, and one tip. It opens the weekly review.

Time

Feature	Copied from	How I'll use it
Recurrence in plain words	Todoist	"Internet bill every month on the 5th" or "lab class every Sun and Tue 10am" sets the repeat rule with no form.
This Evening	Things 3	After-work tasks like "reply to students" sit under Evening, so the daytime view stays clean for job work.
Someday	Things 3	Ideas like "learn Rust" or "refresh portfolio" park in Someday. They never appear on Today; the monthly review checks them.
Do-date vs. deadline	Things 3	A paper revision is due on the 30th but planned for Tuesday. The task shows both, and the deadline warns me only when it's close.
"Does my day fit?" bar	Structured	Planned tasks add up to 9 hours but my free time is 6. Today shows an overflow bar and offers to move the lowest-priority tasks to tomorrow.
Daily goal and vacation mode	Todoist	A goal of 5 tasks a day, with Friday set as a day off. During Eid holidays, vacation mode pauses goals so streaks survive.
Logbook	Things 3	A searchable list of everything I finished. It doubles as a timesheet when either job asks what I did last week.
Grow-a-plant focus	Forest (softened)	During a focus session Tiki waters a plant. If I leave the app it droops, and coming back within a minute revives it. No dead trees, no graveyard.

Feature	Copied from	How I'll use it
Focus statistics	TickTick	Weekly focus minutes per area, e.g. how much deep work Research actually got.
Eisenhower view	TickTick	In weeks where both jobs and teaching deadlines collide, I drag tasks into the urgent/important grid to decide what to drop.
Energy tag	Structured	Tasks marked high or low energy. "Plan my day" puts deep work in the morning and low-energy chores late.

Ease-of-use guardrails learned from these apps

- **Advanced views start switched off.** Eisenhower, energy tags and the shopping list are enabled in Settings only when I want them; TickTick's biggest complaint is feature overload.
- **Colour carries meaning only**, as in Things 3: blue for time, green and coral for money, purple for habits. Everything else stays neutral.
- **Every new feature reuses an existing surface:** payday plan and recap open in Review, events and shopping lists live in Money, This Evening and Someday are sections of Today and Plan.
- **Defaults work out of the box.** Categories, buckets, focus length and quiet hours are pre-filled, so day one needs no setup beyond adding accounts.

AI features (OpenAI API)

AI is used where typing or thinking is the friction: turning messy input into clean records, and turning numbers into plain advice. AI never writes to the database on its own. Every result is a draft I confirm with one tap.

Feature	What it does	Why it exists
Smart quick-add	Turns free text the phone's rules can't parse into a structured task, expense, income or time entry.	Capture stays one text box, however messily I type.

Feature	What it does	Why it exists
Receipt and SMS scan	A photo of a receipt becomes a transaction draft. SMS that no on-phone template can parse also come here, with private details masked.	Covers paper receipts and the odd SMS format the templates miss.
Auto-categorise	Suggests category and area for a new transaction from its note and my recent corrections.	Fewer taps, and it learns my own labels.
Plan my day	Takes my top three, estimates, fixed blocks and bills and proposes a timeline I can drag around.	Turns a list into a realistic day in seconds.
Break it down	Splits a big, vague task into 3–7 concrete subtasks.	Makes scary tasks startable.
Weekly review coach	The server computes the week's numbers; AI writes three observations and one suggestion from them.	Numbers become a short, readable note. AI explains the numbers but never calculates them.
Ask my data	Questions like "how much did I spend on transport in September?" are answered through a fixed set of read-only query functions on the server.	Answers in plain language without building a report screen for every question.
Budget suggestions	At the monthly review, proposes next month's budgets from the last three months of spending.	A sensible starting point instead of guessing.

How it runs

- All AI calls go through the backend under `/ai/*`. The OpenAI API key lives only in the server's environment variables, never in the app.
- The backend uses the official `openai` npm package, which runs on Bun.
- Parsing features use Structured Outputs with a JSON schema built from the same Zod schemas the API already validates with, so replies always fit the database shape.

- "Ask my data" uses function calling. The model can only pick from functions such as `spendByCategory(from, to)` or `hoursByArea(from, to)`. It never writes or runs SQL.
- Model names live in environment variables (a small, cheap model for parsing and categorising; a stronger one for the weekly coach), so I can switch models without a code change.

Guardrails

- **Rules first.** Obvious inputs like "lunch 250" are parsed on the phone and never hit the API. Repeated notes reuse the last category without an AI call.
- **Cost cap.** Every call's token usage is logged in an `ai_usage` table. AI switches off for the rest of the month once a cap I set in Settings is reached.
- **Graceful fallback.** Offline, over the cap, or on an API error, the app falls back to the plain form with nothing lost.
- **Minimum data.** Requests carry only the fields a feature needs. Account numbers are never sent, and names in debts are replaced with placeholders.
- **Off switch.** One toggle in Settings disables all AI features.

This is the one place data leaves my server, so it is a deliberate exception to the "my data, my server" principle and stays opt-in per feature.

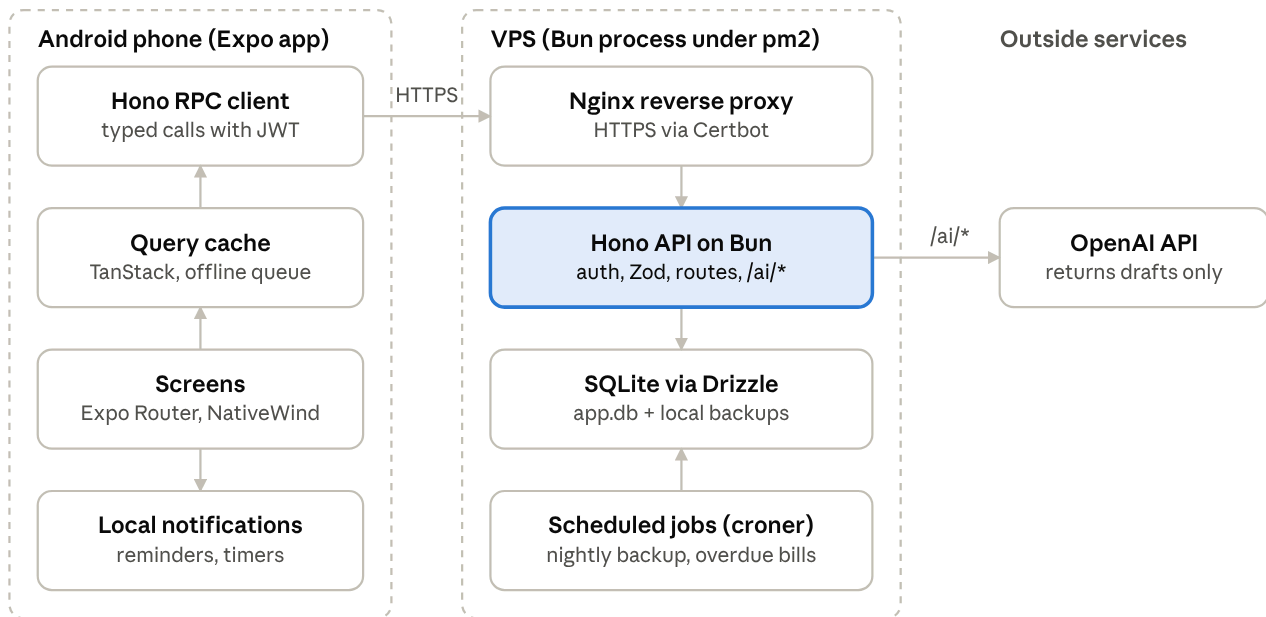
Tech stack and architecture

An Expo app on the phone talks to one Bun process on my VPS, which holds everything in a single SQLite file. No Redis, no Postgres, no queue: at one user's scale, each would add work without adding anything I'd notice.

Layer	Choice	Why this one
App framework	Expo + Expo Router, TypeScript	React skills carry over; file-based routing like the Next.js App Router
Styling	NativeWind	Tailwind classes I already know
Motion and feel	Reanimated, expo-haptics, react-native-svg	Springy animations, taps you can feel, and Tiki's SVG expressions
Data on the phone	TanStack Query with a persisted cache	Instant screens, offline reads, queued offline writes

Layer	Choice	Why this one
API client	Hono RPC client (<code>hc</code>)	End-to-end types from server routes, with no client code to write
Forms and parsing	React Hook Form, Zod, chrono-node	Same Zod schemas as the server; natural-language dates for quick-add
Charts	react-native-gifted-charts	Simple, good-looking bar, line and pie charts
Device features	expo-notifications, expo-secure-store, expo-local-authentication	Reminders, safe token storage, fingerprint lock
Backend runtime	Bun	Fast start, small memory footprint, built-in SQLite and password hashing
Web framework	Hono	Tiny, fast, typed, first-class on Bun
Database	SQLite through <code>bun:sqlite</code> , WAL mode	One file, zero services, easy to back up
ORM	Drizzle	Typed queries and migrations with almost no overhead
Jobs	croner	Time-zone-aware cron inside the same process
AI	<code>openai</code> npm package	Official SDK, works on Bun
Hosting	My VPS: Bun process kept alive by pm2, behind Nginx	pm2 restarts the API on crash and reboot; Nginx with a Certbot certificate handles HTTPS. Run exactly one instance in fork mode, so SQLite has one writer and jobs run once
App builds	EAS Build	Produces the Android APK without a local Android setup

One phone talks to one small server; only AI calls leave it



system architecture · phone, VPS, 2 outside services

Screens read and write through the query cache, which calls the API over HTTPS; only `/ai/*` requests ever leave the server.

Repository layout. One Git repository using Bun workspaces. Bun is the package manager for both apps; the Expo app itself still runs through Metro and Node during development.

```

tick-taka/
  package.json      workspaces config
  apps/
    api/            Hono + Drizzle + SQLite (pm2 config here)
    mobile/         Expo app
  
```

The mobile app imports only the `AppType` type from `apps/api`, which is all the Hono RPC client needs for full type safety.

Data model

Twenty-one SQLite tables cover everything. Balances, totals and streaks are calculated from records, never stored, so they can't drift out of sync.

Conventions for every table

- `id` is a text ULID generated on the phone, so records can be created offline.

- Timestamps are UTC epoch milliseconds (`integer`). The time zone (Asia/Dhaka) lives in `settings` and is applied only for display.
- Habit dates and budget months are local strings (`2026-10-04` , `2026-10`), because "today" is a local idea.
- Money is `integer` in the smallest unit (`amount_minor`), with the currency taken from the account.
- Every table has `created_at` , `updated_at` and `deleted_at` . Deleting sets `deleted_at` , which makes Undo trivial.

Table	Key fields	Notes
settings	key, value (JSON)	Time zone, default currency, default account, AI cap, theme, daily task goal, days off, vacation mode, which advanced views are on
areas	name, emoji, color, sort	Job 1, Job 2, Teaching, Research, Home, Personal
projects	area_id, name, status, sort	status: active, paused, done
tasks	project_id, area_id, parent_id, title, notes, status, priority, do_at, when_slot, deadline_at, energy, has_time, estimate_min, reminder_at, rrule, top3_date, done_at	status: inbox, open, someday, done. when_slot: day or evening. do_at is when I plan to do it; deadline_at is when it must be done. parent_id is for subtasks (one level only)
time_entries	task_id, area_id, project_id, started_at, ended_at, source, billable	source: timer, focus, manual. A running entry has no ended_at
habits	name, emoji, color, schedule, per_week, target_count, archived_at	schedule: daily, weekly, n_per_week
habit_logs	habit_id, date, count	Unique on (habit_id, date)
routines	name, emoji, sort	Morning, Shutdown
routine_steps	routine_id, title, minutes, sort	

Table	Key fields	Notes
accounts	name, type, currency, opening_minor, icon, archived_at	type: cash, bank, mobile_wallet, card, savings
categories	parent_id, name, emoji, kind, budget_type	kind: expense, income. budget_type: fixed, non_monthly, flexible
transactions	type, account_id, to_account_id, amount_minor, to_amount_minor, fee_minor, category_id, area_id, goal_id, debt_id, event_id, recurring_id, note, receipt_path, occurred_at	type: expense, income, transfer, adjustment. to_amount_minor handles cross-currency transfers
budgets	category_id, month, limit_minor, rollover	Unique on (category_id, month)
recurring	kind, name, amount_minor, account_id, category_id, area_id, rrule, next_due_at, remind_days	kind: bill, income
goals	name, emoji, target_minor, deadline, account_id, done_at	Progress = sum of transactions with this goal_id
debts	person, direction, principal_minor, currency, due_at, note, closed_at	direction: owed_to_me, i_owe. Repayments are transactions with this debt_id
ai_usage	feature, model, input_tokens, output_tokens, cost_micros, created_at	Feeds the monthly AI cost cap

The borrowed features add three more tables (and SMS auto-capture adds `sms\_imports`, described in its own section):

Table	Key fields	Notes
events	name, emoji, budget_minor, starts_on, ends_on	A trip or celebration; its spending is every transaction with this event_id

Table	Key fields	Notes
shopping_items	list_name, title, est_minor, checked_at, transaction_id	Ticking off the list at checkout creates one transaction and links it
category_rules	match_text, category_id, area_id	Saved when I correct a category; checked before any AI call

Add indexes on `transactions(occurred_at)` , `transactions(account_id)` , `tasks(status, do_at)` and `time_entries(started_at)` . At one user's volume, SQLite will answer every report in milliseconds.

API design

A plain JSON REST API in Hono, with Hono's typed RPC client (`hc`) on the phone so every call is type-checked end to end without writing a client by hand.

Auth for one user

- No users table. The server holds one password hash (`APP_PASSWORD_HASH`) in its environment, made with `Bun.password.hash` .
- `POST /auth/login` checks the password and returns a JWT signed with `JWT_SECRET` , valid for 30 days (Hono's `hono/jwt` middleware).
- The app stores the token in `expo-secure-store` and sends it as `Authorization: Bearer <token>` on every request.
- Login is limited to 5 attempts per 15 minutes per IP, read from the X-Real-IP header Nginx sets.
- Every body and query is validated with Zod through `@hono/zod-validator` . Errors always return `{ "error": { "code", "message" } }` .

Endpoints

Method and path	Purpose
POST /auth/login	Exchange the password for a token
GET /today?date=	Everything the Today screen needs in one call: agenda, top three, safe-to-spend, habits, running timer, upcoming bills

Method and path	Purpose
GET, POST /tasks · PATCH, DELETE /tasks/:id	Task list with filters (status, area, project, due range) and CRUD
POST /tasks/rescue-overdue	Move all overdue tasks to today, tomorrow or the inbox
GET, POST /areas · /projects · /routines	CRUD for areas, projects and routines (same shape as tasks)
POST /timer/start · POST /timer/stop	Start or stop the single running time entry
GET, POST /time-entries · PATCH, DELETE /time-entries/:id	Manual time entries and edits
GET, POST /habits · PUT /habits/:id/logs/:date	Habits, and check or uncheck a habit for a day
GET, POST /accounts · POST /accounts/:id/balance-check	Accounts with computed balances; log an adjustment to a real balance
GET, POST /transactions · PATCH, DELETE /transactions/:id	Transactions with filters (date range, account, category, area, text search) and cursor paging
GET, POST /categories · GET, PUT /budgets?month=	Categories, and a month's budgets with spent-so-far
GET, POST /recurring · POST /recurring/:id/pay	Bills and expected income; "pay" or "received" creates the transaction and moves next_due_at forward
GET, POST /goals · /debts	Savings goals and debts with progress
GET /insights/summary?from=&to=	Totals by category, area and day; hours by area; effective hourly rates
POST /ai/parse · /ai/receipt · /ai/plan-day · /ai/breakdown · /ai/weekly-review · /ai/ask	AI features; each returns a draft and never saves
GET /sync/changes?since=	Every record updated since a timestamp, for refreshing the phone's offline cache
GET /export	Full JSON export of every table
GET /health	Uptime check, no auth

Notifications, offline, backup and security

Notifications are scheduled on the phone, not pushed from the server. After each sync the app re-schedules them with `expo-notifications`, so there's no push service to run.

Notification	When
Task reminder	At the task's reminder time
Bill due	2 days before, and at 9 am on the due day
Habit nudge	At the habit's chosen time, only if it isn't checked yet
Focus timer	When a work or break period ends
Daily shutdown	9:30 pm
Weekly review	Sunday evening

Quiet hours run from 11 pm to 7 am; anything scheduled then waits until morning.

Server jobs. One small scheduler (the `croner` package, set to Asia/Dhaka) runs inside the same Bun process: marking bills overdue just after midnight and taking the nightly backup. A repeating task creates its next copy when the current one is completed, so no job is needed for that.

Offline behaviour

- TanStack Query keeps a persisted cache on the phone, so every screen opens instantly and still works without internet.
- Changes made offline are queued and sent in order when the connection returns.
- Because IDs are generated on the phone, offline-created records never clash with server records.
- Conflicts resolve by last write wins on `updated_at`, which is safe with a single user.

Backups

- Every night at 3 am the server runs `VACUUM INTO` to write a clean copy of the database to a backups folder next to it, keeping the last 14 days.
- Backups stay on the server; nothing is uploaded to any outside storage.

- Settings has an Export button that downloads the full JSON export to the phone. It is the only copy that lives outside the server, so it's worth tapping now and then.
- Once a month, restore a backup to a local copy to confirm it actually works.
- Keep the backups folder on a different disk or partition from the live database if the VPS has one.

Security

- HTTPS only, through Nginx with a Let's Encrypt certificate from Certbot (renews automatically). The Bun process listens on 127.0.0.1, so only Nginx can reach it.
- App lock with fingerprint or face (`expo-local-authentication`) after 5 minutes in the background.
- Privacy mode: one tap blurs all amounts on screen, handy when using the app in public.
- Receipt images are stored in an uploads folder next to the database and served only to authenticated requests.
- Secrets (`APP_PASSWORD_HASH` , `JWT_SECRET` , `OPENAI_API_KEY`) live in a `.env` file on the server with permissions set to 600, never in Git.
- The SQLite file sits in a data folder outside the repository and is never served over HTTP.

Project setup, step by step

One action per step, from an empty computer to the app running on my phone and the backend live on the VPS.

A. Install the tools

1. Open a terminal (PowerShell on Windows, Terminal on macOS or Linux).
2. Install Bun. On macOS or Linux run `curl -fsSL https://bun.sh/install | bash`. On Windows run `powershell -c "irm bun.sh/install.ps1 | iex"`.
3. Close the terminal.
4. Open a new terminal.
5. Run `bun --version` and check that it prints a version number.
6. Install Node.js LTS from nodejs.org (Expo's tools still need it).
7. Run `node --version` and check that it prints a version number.
8. Run `git --version`. If it fails, install Git from git-scm.com.

9. On the Android phone, install **Expo Go** from the Play Store.

B. Create the project folder

1. Run `mkdir tick-taka`.
2. Run `cd tick-taka`.
3. Run `git init`.
4. Create a file named `package.json` in this folder with this content:

```
{
  "name": "tick-taka",
  "private": true,
  "workspaces": ["apps/*"]
}
```

5. Create a file named `.gitignore` with these three lines:

```
node_modules
apps/api/data
.env
```

6. Run `mkdir apps`.

C. Create the backend (Bun + Hono + SQLite)

1. Run `cd apps`.
2. Run `bun create hono@latest api`.
3. When it asks for a template, choose **bun**.
4. When it asks to install dependencies, choose **Yes**, then choose **bun**.
5. Run `cd api`.
6. Run `bun add drizzle-orm zod @hono/zod-validator`.
7. Run `bun add -d drizzle-kit`.
8. Run `mkdir -p src/db data`.
9. Create `src/db/schema.ts` with this first table (the rest follow the Data model section):

```
import { sqliteTable, text, integer } from "drizzle-orm/sqlite-core";

export const accounts = sqliteTable("accounts", {
  id: text("id").primaryKey(),
  name: text("name").notNull(),
  type: text("type").notNull(),
  currency: text("currency").notNull().default("BDT"),
  openingMinor: integer("opening_minor").notNull().default(0),
  createdAt: integer("created_at").notNull(),
  updatedAt: integer("updated_at").notNull(),
  deletedAt: integer("deleted_at"),
});
```

10. Create `drizzle.config.ts` in the `api` folder:

```
import { defineConfig } from "drizzle-kit";

export default defineConfig({
  dialect: "sqlite",
  schema: "./src/db/schema.ts",
  out: "./drizzle",
  dbCredentials: { url: "./data/app.db" },
});
```

11. Run `bunx drizzle-kit generate`. A `drizzle` folder with a migration file appears.

12. Create `src/db/client.ts`:

```
import { Database } from "bun:sqlite";
import { drizzle } from "drizzle-orm/bun-sqlite";
import { migrate } from "drizzle-orm/bun-sqlite/migrator";

const sqlite = new Database(process.env.DB_PATH ?? "./data/app.db", {
  create: true });
sqlite.exec("PRAGMA journal_mode = WAL;");
sqlite.exec("PRAGMA foreign_keys = ON;");

export const db = drizzle({ client: sqlite });
migrate(db, { migrationsFolder: "./drizzle" });
```

13. Open `src/index.ts`.

14. Replace everything in it with:

```
import { Hono } from "hono";
import { db } from "../db/client";
import { accounts } from "../db/schema";

const app = new Hono();

app.get("/health", (c) => c.json({ ok: true }));
app.get("/accounts", async (c) => c.json(await db.select().from(accounts)));

export default {
  port: Number(process.env.PORT ?? 3000),
  hostname: process.env.HOST ?? "0.0.0.0",
  fetch: app.fetch,
};
```

15. Run `bun run dev`.
16. Open `http://localhost:3000/health` in a browser and check that it shows `{"ok":true}`.
17. Leave this terminal running.

D. Create the mobile app (Expo)

1. Open a second terminal.
2. Go to the `tick-taka/apps` folder with `cd`.
3. Run `bunx create-expo-app@latest mobile`.
4. Run `cd mobile`.
5. Run `bunx expo start`.
6. Make sure the phone and the computer are on the same Wi-Fi.
7. Open Expo Go on the phone.
8. Scan the QR code shown in the terminal.
9. Check that the starter app opens on the phone.
10. Press `Ctrl + C` in the terminal to stop it.
11. Run `bunx expo install expo-notifications expo-secure-store expo-local-authentication expo-haptics react-native-reanimated`.
12. Run `bun add @tanstack/react-query hono zod react-hook-form chrono-node ulid`.

13. Set up NativeWind by following the Expo installation guide on nativewind.dev exactly; its config files change between versions.

E. Connect the phone to the local backend

1. Find the computer's local IP address. Windows: run `ipconfig` and read **IPv4 Address**. macOS: run `ipconfig getifaddr en0`. Linux: run `hostname -I`.
2. In `apps/mobile`, create a file named `.env`.
3. Put one line in it, using your own IP:
`EXPO_PUBLIC_API_URL=http://192.168.0.10:3000`.
4. Make sure the backend from part C is still running.
5. Run `bunx expo start` again so it reads the `.env` file.
6. In the app, call ``${process.env.EXPO_PUBLIC_API_URL}/health`` and check the response.

F. Deploy the backend to the VPS with pm2 and Nginx

pm2 keeps the Bun API running and restarts it after a crash or reboot. The existing Nginx sits in front and handles HTTPS. Run exactly **one** pm2 instance in fork mode: SQLite wants a single writer, and the scheduled jobs must not run twice.

On your computer

1. In `apps/api`, create a file named `ecosystem.config.cjs`:

```
const os = require("os");

module.exports = {
  apps: [
    {
      name: "tick-taka-api",
      script: "src/index.ts",
      cwd: __dirname,
      interpreter: os.homedir() + "/.bun/bin/bun",
      exec_mode: "fork",
      instances: 1,
      max_memory_restart: "300M",
    },
  ],
};
```

2. Make the password hash: run `bun -e 'Bun.password.hash("your-password").then(h => console.log(btoa(h)))'`. It is base64-encoded because Bun expands `$` signs inside `.env` files and the raw hash is full of them; decode it with `atob()` in the login route.
3. Copy the output somewhere safe.
4. Make the JWT secret: run `bun -e 'console.log(crypto.randomUUID() + crypto.randomUUID())'`.
5. Copy that output somewhere safe.
6. Create a new **private** repository on GitHub.
7. In the `tick-taka` folder, run `git add .`.
8. Run `git commit -m "Initial setup"`.
9. Run `git branch -M main`.
10. Run `git remote add origin <your-repo-url>`.
11. Run `git push -u origin main`.
12. In your DNS settings, add an A record for a subdomain (for example `tt-api.bipulhf.dev`) pointing to the VPS IP.

On the VPS (Ubuntu)

1. Connect with `ssh <user>@<vps-ip>`.
2. Install Bun: run `curl -fsSL https://bun.sh/install | bash`.
3. Run `source ~/.bashrc`.
4. Run `bun --version` to check it works.
5. Add the Node.js LTS repository: run `curl -fsSL https://deb.nodesource.com/setup_lts.x | sudo -E bash -`. `pm2` itself runs on Node.
6. Run `sudo apt install -y nodejs`.
7. Run `sudo npm install -g pm2`.
8. Run `git clone <your-repo-url> ~/tick-taka`. Use a GitHub personal access token or deploy key when it asks for credentials.
9. Run `cd ~/tick-taka`.
10. Run `bun install`.
11. Run `mkdir -p ~/tick-taka-data/backups ~/tick-taka-data/uploads`.
12. Run `nano apps/api/.env`.

13. Type these lines, using your own values and username:

```
HOST=127.0.0.1
PORT=3000
DB_PATH=/home/<user>/tick-taka-data/app.db
APP_PASSWORD_HASH=<the base64 hash>
JWT_SECRET=<the secret>
OPENAI_API_KEY=<your key>
```

14. Press `Ctrl + O`, then `Enter` to save.

15. Press `Ctrl + X` to close nano.

16. Run `chmod 600 apps/api/.env`.

17. Run `cd apps/api`.

18. Run `pm2 start ecosystem.config.cjs`.

19. Run `pm2 status` and check that `tick-taka-api` shows **online**.

20. Run `curl http://127.0.0.1:3000/health` and check that it prints `{"ok":true}`.

21. Run `pm2 save`.

22. Run `pm2 startup`.

23. Copy the `sudo ...` command it prints and run it, so the API starts again after a reboot.

24. Run `pm2 install pm2-logrotate` so log files don't fill the disk.

HTTPS with Nginx and Certbot

1. Run `sudo nano /etc/nginx/sites-available/tick-taka`.

2. Paste this server block, using your own subdomain:

```
server {
    listen 80;
    server_name tt-api.bipulhf.dev;

    client_max_body_size 10M;

    location / {
        proxy_pass http://127.0.0.1:3000;
        proxy_http_version 1.1;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;
    }
}
```

3. Press `Ctrl + O`, then `Enter` to save.
4. Press `Ctrl + X` to close nano.
5. Run `sudo ln -s /etc/nginx/sites-available/tick-taka /etc/nginx/sites-enabled/`.
6. Run `sudo nginx -t` and check that it says the syntax is ok and the test is successful.
7. Run `sudo systemctl reload nginx`.
8. Run `certbot --version`. If it fails, run `sudo apt install -y certbot python3-certbot-nginx`.
9. Run `sudo certbot --nginx -d tt-api.bipulhf.dev`.
10. If Certbot asks whether to redirect HTTP to HTTPS, choose redirect.
11. Run `sudo certbot renew --dry-run` and check that it reports success, which confirms automatic renewal works.
12. If the firewall is on, run `sudo ufw allow 'Nginx Full'`.
13. Open `https://tt-api.bipulhf.dev/health` in a browser and check that it shows `{"ok":true}`.

`client_max_body_size 10M` lets receipt photos through; Nginx rejects anything over 1 MB by default. The `X-Real-IP` header is what the login rate limit reads, so keep it in the block.

Deploying an update later

1. On your computer, commit and run `git push`.

2. On the VPS, run `cd ~/tick-taka`.
3. Run `git pull`.
4. Run `bun install`.
5. Run `pm2 restart tick-taka-api`. Database migrations run automatically on startup.
6. Run `pm2 logs tick-taka-api --lines 50` to check for errors.

G. Install the app on the phone

1. Create a free account at expo.dev.
2. In `apps/mobile`, run `bunx eas-cli login`.
3. Run `bunx eas-cli build:configure` and choose **Android**.
4. Open the new `eas.json` file.
5. Inside the `preview` profile, add `"android": { "buildType": "apk" }`.
6. Inside the same `preview` profile, add `"env": { "EXPO_PUBLIC_API_URL": "https://<your-subdomain>" }`.
7. Run `bunx eas-cli build -p android --profile preview`.
8. Wait for the build to finish and open the link it prints.
9. Download the APK on the phone.
10. Allow installing from this source when Android asks.
11. Install and open Tick & Taka.

Build checklist

All 63 features ship together in one release, with no phases. Tick each one off when it works end to end on the phone. The foundation comes first only because every other feature reads from it.

Foundation (app-wide)

- ☐ Login, JWT and SQLite schema with migrations
- ☐ Deployment with pm2 and Nginx
- ☐ Offline cache and queued writes
- ☐ Local notifications and quiet hours
- ☐ Nightly backup and Export
- ☐ Fingerprint app lock and privacy mode
- ☐ Tiki, sparks, levels and streak freezes

Capture (1)

- ☐ Quick-add sheet with on-phone parser

Time (22)

- ☐ Areas and projects
- ☐ Inbox
- ☐ Tasks
- ☐ Top three
- ☐ Week planner
- ☐ Focus timer
- ☐ Time tracking
- ☐ Habits
- ☐ Routines
- ☐ Calendar view
- ☐ Overdue rescue
- ☐ Recurrence in plain words
- ☐ This Evening
- ☐ Someday
- ☐ Do-date vs. deadline
- ☐ "Does my day fit?" bar
- ☐ Daily goal and vacation mode
- ☐ Logbook
- ☐ Grow-a-plant focus
- ☐ Focus statistics
- ☐ Eisenhower view
- ☐ Energy tag

Money (24)

- ☐ Accounts
- ☐ Transactions
- ☐ Categories
- ☐ Budgets
- ☐ Safe to spend

- ☐ Bills and subscriptions
- ☐ Expected income
- ☐ Savings goals
- ☐ Debts and lending
- ☐ Balance check
- ☐ Reports
- ☐ SMS auto-capture and review panel
- ☐ Payday plan
- ☐ Three-bucket budgets
- ☐ Pace alert
- ☐ Calculator keypad
- ☐ Money calendar
- ☐ Home-screen widget
- ☐ Debt payoff forecast
- ☐ Events
- ☐ Priced shopping list
- ☐ Learn-from-corrections categories
- ☐ Subscription spotter
- ☐ Weekly recap card

Time and money together (8)

- ☐ Effective hourly rate
- ☐ Cost in hours
- ☐ One timeline
- ☐ Area dashboard
- ☐ Goals become tasks
- ☐ Daily shutdown
- ☐ Weekly review
- ☐ Monthly review

AI (8)

- ☐ Smart quick-add
- ☐ Receipt and SMS scan

- ☐ Auto-categorise
- ☐ Plan my day
- ☐ Break it down
- ☐ Weekly review coach
- ☐ Ask my data
- ☐ Budget suggestions

Deliberately not building

- Bank or bKash API sync. There's no reliable personal API; reading SMS on the phone covers it.
- Multiple users, sharing or family accounts.
- A web or desktop version.
- An iOS version. The app is Android-only.
- Google Calendar sync, until the in-app calendar proves it isn't enough.
- Free-form tags beyond areas and projects.
- Any AI feature that saves records without my confirmation.